

DB

If you have limited time, revise in this order:

1. SQL queries
 2. JOINS
 3. GROUP BY / HAVING / aggregates
 4. Primary key / Foreign key / constraints
 5. Indexes
 6. Transactions + ACID
 7. Normalization
 8. Subqueries / CTEs
 9. Window functions
 10. Isolation levels / concurrency
 11. DELETE vs TRUNCATE vs DROP
 12. WHERE vs HAVING
 13. SQL vs NoSQL
 14. Database design
 15. Query optimization
 16. Stored procedures / views / triggers
 17. Locks / deadlocks
 18. Pagination
 19. Transactions in backend applications
 20. PostgreSQL-specific concepts
-

PART 1 - DATABASE FUNDAMENTALS

1. What is a Database?

A database is an organized collection of data that can be stored, accessed, modified, and managed efficiently.

Example:

```
Users
-----
id | name  | email
1  | Ali   | ali@gmail.com
2  | Sara  | sara@gmail.com
```

2. DBMS vs RDBMS

DBMS

Database Management System.

Examples:

- MongoDB
- Redis
- some hierarchical databases

RDBMS

Relational Database Management System.

Data is organized into **tables with relationships**.

Examples:

- PostgreSQL
- MySQL
- SQL Server
- Oracle

Interview answer

An RDBMS stores structured data in tables and allows relationships between tables using keys such as primary keys and foreign keys. It typically supports SQL and transactional guarantees.

3. What is a Table?

A table contains:

- rows
- columns

Example:

```
CREATE TABLE users (  
    id INT,  
    name VARCHAR(100),  
    email VARCHAR(255)  
);
```

4. Row vs Column

Row

One record.

```
1 | Ali | ali@gmail.com
```

Column

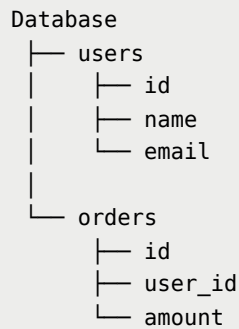
One attribute.

```
id  
name  
email
```

5. Schema

A schema describes the structure of a database.

For example:



PART 2 - SQL COMMANDS

You need to know these categories.

DDL

Data Definition Language.

Used to define database structure.

```
CREATE
ALTER
DROP
TRUNCATE
```

Example:

```
CREATE TABLE users (...);
```

DML

Data Manipulation Language.

```
INSERT  
UPDATE  
DELETE
```

Example:

```
INSERT INTO users (name)VALUES ('Ali');
```

DQL

Data Query Language.

```
SELECT
```

DCL

Data Control Language.

```
GRANT  
REVOKE
```

TCL

Transaction Control Language.

```
COMMIT  
ROLLBACK  
SAVEPOINT
```

PART 3 - CRUD

INSERT

```
INSERT INTO users (name, email)VALUES ('Ali','ali@gmail.com');
```

Multiple:

```
INSERT INTO users (name, email)VALUES  
('Ali','ali@gmail.com'),  
('Sara','sara@gmail.com');
```

SELECT

```
SELECT *FROM users;
```

Specific columns:

```
SELECT name, email FROM users;
```

UPDATE

```
UPDATE users  
SET email='new@gmail.com'  
WHERE id=1;
```

Dangerous:

```
UPDATE users  
SET email='new@gmail.com';
```

This updates **every row**.

DELETE

```
DELETE FROM users  
WHERE id=1;
```

Again:

```
DELETE FROM users;
```

deletes every row.

PART 4 - SQL DATA TYPES

Common ones:

Numeric

```
INTEGER  
BIGINT  
DECIMAL  
NUMERIC  
REAL  
DOUBLE PRECISION
```

String

```
CHAR  
VARCHAR  
TEXT
```

Date/time

```
DATE  
TIME  
TIMESTAMP  
TIMESTAMPZ
```

Boolean

BOOLEAN

PostgreSQL-specific

UUID
JSON
JSONB
ARRAY

PART 5 - PRIMARY KEY

A primary key uniquely identifies each row.

```
CREATE TABLE users (  
    id BIGINT PRIMARY KEY,  
    name TEXT  
);
```

Properties:

- unique
- cannot be NULL
- one primary key constraint per table

6. Can a table have multiple primary keys?

No.

But a primary key can contain multiple columns.

That's called a:

| Composite Primary Key

Example:


```
PRIMARY KEY (student_id, course_id)
```

7. UNIQUE

Ensures values are unique.

```
email VARCHAR(255) UNIQUE
```

Difference:

PRIMARY KEY	UNIQUE
Identifies row	Enforces uniqueness
Cannot NULL	NULL behavior depends on DB
One PK constraint	Multiple UNIQUE constraints

8. FOREIGN KEY

Maintains relationships between tables.

```
CREATE TABLE orders (  
    id BIGINT PRIMARY KEY,  
    user_id BIGINT REFERENCES users(id)  
);
```

Here:

```
users.id  
  ↑  
  |  
orders.user_id
```

9. Referential Integrity

Foreign keys ensure that relationships remain valid.

For example, you shouldn't have:

```
orders.user_id=9999
```

if user 9999 doesn't exist.

10. Constraints

Know all of these:

```
PRIMARY KEY  
FOREIGN KEY  
UNIQUE  
NOT NULL  
CHECK  
DEFAULT
```

Example:

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  email TEXT UNIQUE NOT NULL,  
  age INT CHECK(age>=18),  
  country TEXT DEFAULT 'Pakistan'  
);
```

PART 6 - NULL

This is **very frequently tested**.

NULL means:

| Missing/unknown/not applicable value.

It is NOT:

```
0
''
FALSE
```

Wrong

```
WHERE email=NULL
```

Correct

```
WHERE email IS NULL;
```

And:

```
WHERE email IS NOT NULL;
```

11. COALESCE

Returns first non-null value.

```
SELECT COALESCE(phone, 'No phone')
FROM users;
```

Example:

```
phone
-----
NULL
123456
```

Result:

No phone
123456

PART 7 - WHERE

Filters rows **before** grouping.

```
SELECT * FROM employees  
WHERE salary > 100000;
```

Operators:

= ! < > <= >=

12. AND / OR / NOT

```
WHERE salary > 50000  
AND department = 'IT';
```

13. BETWEEN

```
WHERE salary  
BETWEEN  
50000  
AND  
100000;
```

Usually inclusive.

14. IN

Instead of:

```
WHERE department='IT'  
OR department='HR'  
OR department='Finance'
```

use:

```
WHERE department IN ('IT','HR','Finance');
```

15. LIKE

Pattern matching.

```
WHERE name LIKE 'A%';
```

Means starts with A.

```
'A%'
```

starts with A.

```
'%A'
```

ends with A.

```
'%ali%'
```

contains ali.

16. ILIKE

PostgreSQL case-insensitive LIKE.

```
WHERE name ILIKE '%ali%';
```

PART 8 - ORDER BY

```
SELECT * FROM employees  
ORDER BY salary DESC;
```

Ascending:

```
ASC
```

Descending:

```
DESC
```

17. LIMIT

```
SELECT * FROM users  
LIMIT 10;
```

18. OFFSET

```
SELECT * FROM users  
LIMIT 10  
OFFSET 20;
```

Gets rows after first 20.

PART 9 - DISTINCT

Remove duplicates.

```
SELECT DISTINCT department
FROM employees;
```

PART 10 - AGGREGATE FUNCTIONS

You MUST know:

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

Example:

```
SELECT AVG(salary)
FROM employees;
```

19. COUNT(*) vs COUNT(column)

Important interview question.

```
COUNT(*)
```

counts rows.

```
COUNT(email)
```

counts non-null email values.

Example:

id	email
1	a
2	NULL
3	c

`COUNT(*)=3`

`COUNT(email)=2`

PART 11 - GROUP BY

Groups rows.

Example:

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

Result:

department	avg_salary
IT	120000
HR	90000
Finance	150000

20. WHERE vs HAVING

Very common interview question.

WHERE

Filters individual rows **before grouping**.

HAVING

Filters groups **after grouping**.

Example:

```
SELECT department, AVG(salary)
FROM employees
WHERE salary > 50000
GROUP BY department
HAVING AVG(salary) > 100000;
```

Logical order:

```
FROM
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
ORDER BY
↓
LIMIT
```

PART 12 - JOINS

This is one of the **most important SQL topics**.

Suppose:

users

id	name
1	Ali
2	Sara
3	John

orders

id	user_id	amount
1	1	500
2	1	700
3	2	300

21. INNER JOIN

Returns matching rows.

```
SELECT u.name, o.amount
FROM users u
INNER JOIN orders o
ON u.id = o.user_id;
```

Result:

Ali	500
Ali	700
Sara	300

John isn't returned because he has no order.

22. LEFT JOIN

Returns **all rows from left table**.

```
SELECT u.name, o.amount
FROM users u
LEFT JOIN orders o
ON u.id = o.user_id;
```

Result:

```
Ali 500
Ali 700
Sara 300
John NULL
```

Interview rule:

| LEFT JOIN = everything from left + matching data from right.

23. RIGHT JOIN

Opposite.

```
SELECT * FROM users u
RIGHT JOIN orders o
ON u.id = o.user_id;
```

All orders appear.

Interview rule:

| RIGHT JOIN = everything from right + matching data from left.

24. FULL OUTER JOIN

Returns:

```
everything from both tables
```

Matching where possible.

25. CROSS JOIN

Cartesian product.

If:

```
users = 3 rows  
products = 4 rows
```

Then:

```
3 × 4 = 12 rows
```

```
SELECT * FROM users  
CROSS JOIN products;
```

26. SELF JOIN

A table joins itself.

Example employee-manager relationship:

```
employees  
id | name | manager_id
```

Query:

```
SELECT  
    e.name AS employee,  
    m.name AS manager  
FROM employees e  
LEFT JOIN employees m  
ON e.manager_id = m.id;
```

27. Most Important JOIN Question

Find users who have never placed an order.

```
SELECT u.*  
FROM users u  
LEFT JOIN  
orders o  
ON u.id = o.user_id  
WHERE o.id IS NULL;
```

This pattern is extremely important.

PART 13 - SUBQUERIES

A query inside another query.

Example:

```
SELECT *  
FROM employees  
WHERE salary >  
(SELECT AVG(salary)  
FROM employees  
);
```

Find employees earning above average.

28. IN subquery

```
SELECT *  
FROM users  
WHERE id IN (SELECT user_id FROM orders);
```

29. EXISTS

Checks whether a subquery returns a row.

```
SELECT * FROM users u
WHERE EXISTS (SELECT 1
FROM orders o
WHERE o.user_id = u.id);
```

Conceptually:

| Does at least one matching order exist?

30. EXISTS vs IN

Know the conceptual difference.

EXISTS checks existence.

IN compares against a set of values.

For large correlated existence checks, **EXISTS** can often be a better expression and may optimize efficiently depending on the database/query.

Don't claim " **EXISTS** is always faster" in an interview.

PART 14 - CTE

Common Table Expression.

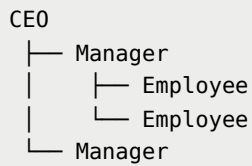
```
WITH high_salary AS
(SELECT * FROM employees
WHERE salary > 100000
)
SELECT * FROM high_salary;
```

Makes complex queries easier to read.

31. Recursive CTE

Used for hierarchical data.

Example:



Recursive CTE can traverse this hierarchy.

PART 15 - WINDOW FUNCTIONS

Very important for modern SQL interviews.

Unlike `GROUP BY`, window functions **don't collapse rows**.

Example:

```
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER (
        PARTITION BY department
    ) AS dept_avg
FROM employees;
```

Each employee remains a row.

32. ROW_NUMBER

```
SELECT
    name,
    salary,
    ROW_NUMBER() OVER ( ORDER BY salaryDESC
```

```
) AS rank  
FROM employees;
```

Produces:

```
Ali    150k  1  
Sara   140k  2  
John   130k  3
```

33. RANK vs DENSE_RANK

Suppose:

```
salary  
100  
100  
90  
80
```

RANK

```
1  
1  
3  
4
```

DENSE_RANK

```
1  
1  
2  
3
```

34. PARTITION BY

Similar conceptually to grouping, but it doesn't collapse rows.

```
RANK() OVER (  
    PARTITION BY department  
    ORDER BY salary DESC  
)
```

Ranks employees **within each department**.

```
DENSE_RANK() OVER (  
    PARTITION BY department  
    ORDER BY salary DESC  
)
```

PART 16 - MOST ASKED SQL QUESTIONS

Now memorize these patterns.

Q1. Find second highest salary.

Method 1

```
SELECT MAX(salary)  
FROM employees  
WHERE salary < (SELECT MAX(salary) FROM employees  
);
```

Method 2 - OFFSET

```
SELECT DISTINCT salary  
FROM employees  
ORDER BY salary DESC  
LIMIT 1 OFFSET 1;
```

Method 3 - DENSE_RANK

```
SELECT salary
FROM (SELECT
      salary,
      DENSE_RANK() OVER (
        ORDER BY salary DESC
      ) AS rnk
      FROM employees
    ) x
WHERE rnk=2;
```

The third approach is especially useful when handling duplicate salaries.

SQL, `x` is an alias (a temporary nickname) for the subquery defined inside the parentheses.

Why is it needed?

1. **SQL Syntax Requirement:** Standard SQL requires every subquery (also called a derived table or inline view) in the `FROM` clause to have a designated name. Without `x` (or any valid identifier name), the database parser will raise a syntax error like `subquery in FROM must have an alias`.
2. **Column Reference:** The outer query selects `salary` and filters by `WHERE rnk = N`. By assigning `x` to the subquery, SQL knows that `salary` and `rnk` belong to the dataset generated by `x`.

Q2. Find Nth highest salary

```
SELECT salary
FROM (SELECT
      salary,
      DENSE_RANK() OVER ( ORDER BY salary DESC
    ) AS rnk
      FROM employees
```

```
) x  
WHERE rnk= N;
```

Q3. Find duplicate emails

```
SELECT email, COUNT(*)  
FROM users  
GROUP BY email  
HAVING COUNT(*)>1;
```

Q4. Find employees with salary above department average

```
SELECT employee_id, first_name, last_name, salary, department  
_id  
FROM employees e  
WHERE salary > (  
SELECT AVG(salary)  
FROM employees  
WHERE department_id = e.department_id  
);
```

```
SELECT *  
FROM (  
    SELECT  
        e.*,  
        AVG(salary) OVER (  
            PARTITION BY department_id  
        ) AS dept_avg  
    FROM employees e
```

```
) x  
WHERE salary > dept_avg;
```

Q5. Find top 3 salaries per department

```
SELECT *  
FROM (SELECT  
      e.*,  
      DENSE_RANK() OVER (  
        PARTITION BY department_id  
        ORDER BY salary DESC  
      ) AS rnk  
      FROM employees e  
    ) x  
WHERE rnk<=3;
```

Q6. Find users who never ordered

```
SELECT u.*  
FROM users u  
LEFT JOIN orders o  
ON u.id = o.user_id  
WHERE o.id IS NULL;
```

Q7. Find customers with more than 5 orders

```
SELECT user_id, COUNT(*) AS order_count  
FROM orders  
GROUP BY user_id  
HAVING COUNT(*)>5;
```

Q8. Find maximum salary in each department

```
SELECT department_id, MAX(salary)
FROM employees
GROUP BY department_id;
```

Q9. Find employee with maximum salary

```
SELECT *
FROM employees
WHERE salary =
(SELECT MAX(salary)
FROM employees);
```

Q10. Delete duplicates

Concept depends on which row should survive.

A common PostgreSQL approach:

```
DELETE FROM users a
USING users b
WHERE a.id > b.id
AND a.email = b.email;
```

Keeps the smallest ID.

The `USING` clause in SQL is a simplified syntax for **joining tables** when both tables share a column with the **exact same name**.

Instead of writing a full `ON` condition, `USING` acts as a shortcut.

Comparison: **ON** vs **USING**

Standard **ON** syntax:

SQL

```
SELECT *  
FROM employees e  
JOIN departments d  
  ON e.department_id = d.department_id;
```

Shorter **USING** syntax:

SQL

```
SELECT *  
FROM employees  
JOIN departments  
  USING (department_id);
```

Both queries perform the exact same join.

PART 17 - NORMALIZATION

Very common DBMS interview topic.

Normalization organizes data to reduce:

- redundancy
- inconsistency
- update anomalies

35. 1NF

First Normal Form.

Rules:

- atomic values

- no repeating groups

Bad:

```
student | courses
Ali     | DB, OS, AI
```

Better:

```
student | course
Ali     | DB
Ali     | OS
Ali     | AI
```

36. 2NF

Must:

- be in 1NF
- have no partial dependency on part of a composite key

Important mainly when a table has a composite key.

37. 3NF

Must:

- be in 2NF
- have no transitive dependency

Example:

```
employee_id
department_id
department_name
```

If:

```
employee_id → department_id  
department_id → department_name
```

then:

```
employee_id → department_name
```

This creates transitive dependency.

Separate department information.

38. BCNF

Boyce-Codd Normal Form.

More strict than 3NF.

Basic rule:

| Every determinant should be a candidate key.

For most SDE interviews, understand the concept rather than memorizing obscure decomposition examples.

39. Denormalization

Intentionally introducing redundancy to improve read performance.

Example:

Instead of joining:

```
orders  
users
```

you might store:


```
customer_name
```

inside an order record.

Trade-off:

```
less joins
  ↓
faster reads

but

more redundancy
  ↓
harder consistency
```

PART 18 - DATABASE KEYS

Know these.

Primary Key

Unique identifier.

Candidate Key

Any minimal set of columns that can uniquely identify a row.

Super Key

Any set of attributes that uniquely identifies a row, possibly with unnecessary attributes.

Alternate Key

Candidate key not selected as primary key.

Foreign Key

References another table.

Composite Key

Multiple columns form a key.

PART 19 - INDEXES

One of the most important backend interview topics.

An index is a data structure that helps the database find rows faster.

Think:

Without index:

Search entire table

1 → 2 → 3 → 4 → ... → 1 million

With index:

Index

↓

locate relevant rows

↓

fetch data

If an interviewer asks:

"What is an index in a database?"

A strong beginner-friendly answer is:

An index is a data structure that allows the database to find rows faster without scanning the entire table. It is similar to the index of a book: instead of reading every page to find a topic, you use the index to jump directly to the relevant pages.

But that's only the beginning. For interviews, you need to understand **how indexes work, types, B-Trees, composite indexes, clustered vs non-clustered indexes, trade-offs, and query optimization.**

40. Why Do We Need Indexes?

Suppose we have:

```
CREATE TABLE users (  
    id INT,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    age INT  
);
```

And there are:

```
10 rows  
100 rows  
1,000 rows  
1 million rows  
100 million rows
```

You execute:

```
SELECT * FROM users  
WHERE email='kinza@example.com';
```

Without an index, the database may have to check every row:

```
Row 1 → email?  
Row 2 → email?  
Row 3 → email?  
Row 4 → email?  
...  
Row 1,000,000 → email?
```

This is called a:

Full Table Scan

Approximately:

$O(n)$

If there are 1 million rows, potentially 1 million rows need to be examined.

What Happens With an Index?

Create an index:

```
CREATE INDEX idx_users_email  
ON users(email);
```

Now the database has an additional data structure.

Conceptually:

INDEX	
email	→ row location
alice@gmail.com	→ row 25
bob@gmail.com	→ row 91
kinza@example.com	→ row 734
zara@gmail.com	→ row 1200

Now:

```
SELECT * FROM users WHERE email = 'kinza@example.com';
```

The database can search the index instead of scanning the entire table.

Conceptually:

```
Query  
↓  
Index  
↓  
Find email  
↓  
Find corresponding row
```

↓
Return row

This can dramatically improve performance.

41. Real-Life Analogy

Imagine a 1,000-page book.

You want:

| "Database normalization"

Without index

You start:

```
Page 1
Page 2
Page 3
...
Page 500
...
Page 1000
```

With index

At the back:

```
Database normalization → Page 735
```

You immediately jump to page 735.

That's essentially the idea behind database indexing.

How Does a Database Index Actually Work?

This is **very important for interviews**.

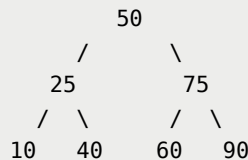
Most relational databases commonly use a structure based on a:

B-Tree / B+ Tree

Don't worry about the name initially.

Think of it as a **sorted tree structure designed to make searching efficient**.

For example:



If you're looking for:

60

You don't inspect every value.

You do:

```
60 > 50
  ↓
  75
  ↓
60 < 75
  ↓
  60
```

So the search is much faster than scanning everything.

Why Not Use a Normal Binary Search Tree?

Because databases deal with **disk/SSD I/O**, not just CPU memory operations.

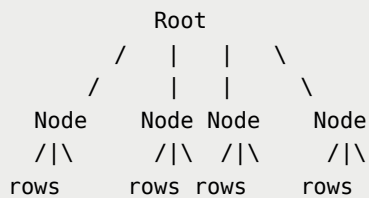
A database may have millions/billions of rows.

B-Trees/B+ Trees are designed to have:

- high branching factor

- relatively small height
- efficient disk/page access

Conceptually:



The tree remains relatively shallow.

This reduces the number of disk/page accesses.

B-Tree vs B+ Tree

Interviewers sometimes ask this.

B-Tree

Data can potentially exist in:

```

internal nodes
+
leaf nodes
  
```

B+ Tree

Typically:

```

Internal nodes
  ↓
keys for navigation

Leaf nodes
  ↓
actual index entries
  
```

And leaf nodes are commonly linked:

```
[10,20,30] → [40,50,60] → [70,80,90]
```

That makes **range queries** very efficient.

For example:

```
SELECT * FROM users WHERE age BETWEEN 20 AND 30;
```

The database can locate the starting point and then traverse the relevant leaf entries.

Big-O of Index Searching

For a B-Tree/B+ Tree index:

```
Search  $\approx O(\log n)$ 
```

Compared to:

```
Full table scan  $\approx O(n)$ 
```

For example:

```
1,000,000 rows
```

A linear scan could potentially inspect:

```
1,000,000
```

entries.

A balanced tree requires dramatically fewer tree levels.

Important interview nuance: Don't say "an index always makes a query $O(\log n)$." Real query performance depends on selectivity, index structure, cache, storage engine, query plan, and whether the optimizer chooses the index.

Creating an Index

Basic syntax:

```
CREATE INDEX index_name ON table_name(column_name);
```

Example:

```
CREATE INDEX idx_users_email ON users(email);
```

Now:

```
SELECT * FROM users WHERE email='kinza@example.com';
```

can potentially use the index.

Indexing a Primary Key

Suppose:

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    name VARCHAR(100)  
);
```

The database generally creates/enforces an index associated with the primary key.

So you normally don't need:

```
CREATE INDEX idx_users_id ON users(id);
```

again.

Unique Index

Suppose email must be unique.

```
CREATE UNIQUE INDEX idx_users_email ON users(email);
```

This does two things:

1. Improves lookup

```
WHERE email= ...
```

2. Enforces uniqueness

You can't have:

```
kinza@example.com  
kinza@example.com
```

twice.

In SQL, you can also express the business rule with:

```
UNIQUE(email)
```

and the database will typically use an appropriate index internally.

The Most Important Trade-Off

Here's where many beginners make a mistake.

They think:

| "Indexes make databases faster, so index every column."

✗ Wrong.

Indexes improve **reads**, but they have costs.

Indexes Consume Storage

Suppose your table contains:

```
100 million rows
```

Adding several indexes means the database must store additional structures.

So:

```
Table  
+  
Index 1  
+  
Index 2  
+  
Index 3  
+  
...
```

requires additional disk space.

Indexes Slow Down INSERT

Suppose:

```
INSERT INTO users  
VALUES (...);
```

Without indexes:

```
Insert row
  ↓
Done
```

With an index:

```
Insert row
  ↓
Update table
  ↓
Update index
```

The database has additional work.

UPDATE Can Also Become More Expensive

Suppose:

```
UPDATE users
SET email='new@example.com'
WHERE id=10;
```

If `email` has an index, the database may need to update the index entry as well.

So:

```
UPDATE table
  +
UPDATE indexes
```

DELETE Can Also Become More Expensive

```
DELETE FROM users
WHERE id=10;
```

The database needs to remove/update relevant index entries.

Therefore:

Indexes improve read performance but add storage and write/maintenance overhead.

MEMORIZE THIS FOR INTERVIEWS.

When Should You Create an Index?

Good candidates are columns frequently used in:

WHERE

```
SELECT * FROM users
WHERE email='x';
```

JOIN

```
SELECT * FROM orders o
JOIN users u
ON o.user_id = u.id;
```

ORDER BY

```
SELECT*FROM usersORDERBY created_at;
```

GROUP BY

Depending on the query and database:

```
SELECT department,COUNT(*)
FROM employees
GROUP BY department;
```

High-Selectivity Columns

This is an important interview concept.

Selectivity

Roughly:

| How well a column distinguishes one row from another.

Consider:

```
id
```

Values:

```
1  
2  
3  
4  
5  
...
```

Very high selectivity.

But:

```
gender
```

might contain:

```
Male  
Female  
Male  
Female  
...
```

Low selectivity.

An index on a highly selective column is often more useful for equality filtering.

But don't blindly say:

“Never index low-cardinality columns.”

That's too absolute.

A low-cardinality index can still be useful depending on:

- data distribution
 - table size
 - query predicates
 - combined/composite indexes
 - database optimizer
 - partitioning/storage engine
-

Cardinality

You will hear:

“Index based on cardinality.”

Cardinality means approximately:

Number of distinct values in a column.

Example:

```
id:
```

```
1  
2  
3  
4  
5
```

Cardinality = 5.

For:

```
gender:
```

```
Male  
Female  
Male  
Male  
Female
```

Cardinality = 2.

High cardinality often means more distinct values.

Single-Column Index

Example:

```
CREATE INDEX idx_users_name ON users(name);
```

This indexes one column.

Useful for:

```
WHERE name='Kinza'
```

Composite Index

Very important for interviews.

Suppose we frequently query:

```
SELECT * FROM employees  
WHERE department='Engineering' AND salary>100000;
```


We could create:

```
CREATE INDEX idx_employee_dept_salary ON employees(department, salary);
```

This is called a:

Composite index / multi-column index

It contains multiple columns.

Column Order Matters!

This is one of the most frequently asked indexing questions.

Suppose:

```
CREATE INDEX idx_employee ON employees(department, salary);
```

The order is:

```
department → salary
```

Not:

```
salary → department
```

This matters because of the **leftmost-prefix principle** in systems such as MySQL's B-tree indexes.

Leftmost Prefix

Index:

```
(department, salary)
```

Can generally help with:

```
WHERE department='Engineering'
```

and:

```
WHERE department='Engineering'AND salary>100000
```

But a query only on:

```
WHERE salary>100000
```

usually cannot use that composite B-tree index as efficiently because it skips the leading column.

Think:

```
(department, salary)
      ↑
    first column
```

The database organizes the index primarily around `department`.

Example

Imagine:

Department	Salary
Engineering	80k
Engineering	100k
Engineering	120k
HR	70k

HR	90k
Sales	80k
Sales	110k

The index is effectively organized around:

```
department
  ↓
salary
```

So finding:

```
Engineering
```

is easy.

Then within Engineering:

```
salary
```

can be searched.

But asking:

```
salary = 100k
```

without department doesn't align as naturally with this ordering.

How Do You Decide Composite Index Column Order?

Common interview answer:

Put columns that are commonly used to narrow the search earlier, considering selectivity and the actual workload/query patterns.

But there is **no universal rule** like:

| "Always put the highest-cardinality column first."

You need to consider:

- query patterns
- equality predicates
- range predicates
- selectivity
- sorting/grouping
- joins

For example:

```
WHERE department= ?AND salary> ?
```

A common useful index is:

```
(department, salary)
```

because `department` is an equality condition followed by a range condition.

Clustered Index

Very common interview topic.

A **clustered index** determines how table data is physically/logically organized around the index key, depending on the database engine.

In systems such as SQL Server, a clustered index means the table's rows are stored in the order of the clustered index key.

Conceptually:

Clustered index

10 → row
20 → row
30 → row
40 → row
50 → row

The leaf level contains the actual table rows.

Non-Clustered Index

A non-clustered index is a separate structure that contains:

indexed value
↓
pointer / row locator
↓
actual row

Conceptually:

Index

email
↓
row locator
↓
table row

So searching may involve:

Index
↓
Find matching entry
↓
Fetch table row

Clustered vs Non-Clustered

Remember this:

Clustered	Non-clustered
Organizes table data around index key	Separate index structure
Leaf level contains table data in systems like SQL Server	Leaf level contains keys + row locators
Typically one per table	Can have multiple
Useful for range access	Useful for many different lookup patterns

Important caveat

Terminology differs between database systems.

For example:

- SQL Server explicitly distinguishes clustered/nonclustered indexes.
- InnoDB's primary key is clustered.
- PostgreSQL does not have SQL Server-style permanently clustered indexes; `CLUSTER` is a physical reorganization operation, not the same indexing model.

For interviews, **always consider which DBMS you're discussing.**

Covering Index

Another important concept.

Suppose:

```
SELECT name, salary
FROM employees
WHERE department='Engineering';
```

And you have an index that contains all required information:

```
(department, name, salary)
```

The database may be able to answer the query directly from the index without fetching the full table row.

This is called a:

Covering index

Conceptually:

```
Query
↓
Index
↓
Everything needed is already there
↓
Return result
```

This can be very fast.

Index-Only Scan

Related concept:

If the database can satisfy the query entirely from the index, it may perform an index-only scan.

PostgreSQL commonly uses this terminology.

What Is a Full Table Scan?

Example:

```
SELECT * FROM users WHERE age=21;
```

Without a useful index:

```
Scan row 1
Scan row 2
Scan row 3
...
```

This is:

| Full table scan / sequential scan

It isn't automatically bad.

For example, if you're returning a huge percentage of the table, scanning the table can actually be cheaper than using an index and then fetching many rows.

Very important interview point:

| The database optimizer does not use an index simply because one exists.

Query Optimizer

The database has a component called the:

| **Query Optimizer**

It examines possible execution strategies.

For example:

```
Query
↓
Optimizer
↓
Should I:
  |— use index?
  |— scan table?
  |— use another index?
  |— use a join strategy?
  |— etc.
```


It estimates costs and chooses an execution plan.

42. EXPLAIN

If you want to know whether an index is being used:

```
EXPLAIN SELECT * FROM users WHERE email='kinza@example.com';
```

Depending on DBMS, you may use:

```
EXPLAIN ANALYZE
```

This helps inspect the execution plan.

For example, you might see:

```
Index Scan
```

instead of:

```
Seq Scan
```

43. Why Would the Database Ignore an Index?

This is a **very good interview question**.

Reason 1 - Too many rows match

Suppose:

```
WHERE gender='Male'
```

and 49% of the table is male.

Using the index might require:

```
Index
↓
Millions of matching rows
↓
Table lookups
```

A sequential scan may be cheaper.

Reason 2 - Small table

If there are only:

```
100 rows
```

scanning the table may be faster than navigating an index.

Reason 3 - Function on indexed column

Suppose:

```
CREATE INDEX idx_users_email
ON users(email);
```

Then:

```
WHERE LOWER(email)='kinza@example.com'
```

may prevent use of the plain index depending on DBMS/query structure.

A functional index may be needed:

```
CREATE INDEX idx_lower_email
ON users(LOWER(email));
```

44. Index and LIKE

Suppose:

```
WHERE name LIKE 'Kin%'
```

A B-tree index can often help because the pattern has a known prefix.

But:

```
WHERE name LIKE '%in%'
```

is generally much harder for a normal B-tree index to use efficiently because the beginning is unknown.

Think:

```
'Kin%'
  ↑
known beginning
```

VS

```
'%in%'
  ↑
unknown beginning
```

For substring/full-text searching, specialized indexes may be more appropriate.

45. Index and NULL

Indexes can generally contain/index NULL values, but exact behavior and optimizer choices vary by database.

Don't say:

| "Indexes don't work with NULL."

That's an oversimplification.

Instead say:

"NULL handling is DBMS-specific, and whether an index is useful for NULL predicates depends on the database and index type."

46. Index and Foreign Keys

Suppose:

```
orders(  
  id,  
  user_id  
)
```

where:

```
user_id → users.id
```

A foreign key column is often a good candidate for an index because it's commonly used in:

```
JOIN
```

and:

```
WHERE
```

For example:

```
CREATE INDEX idx_orders_user_id  
ON orders(user_id);
```

Important:

A foreign key constraint does not universally mean the database automatically creates an index on the child column.

This depends on the DBMS.

47. Indexes and JOINS

Suppose:

```
SELECT *  
FROM orders o  
JOIN users u  
ON o.user_id = u.id;
```

Indexes can help the database find matching rows efficiently.

Usually:

```
users.id
```

is indexed because it's a primary key.

It can also be useful to index:

```
orders.user_id
```

depending on query workload.

48. Index for ORDER BY

Suppose:

```
SELECT *  
FROM employees  
ORDER BY salary;
```

An appropriate index on:

```
salary
```

may help avoid a separate expensive sort in some cases.

But again:

| The optimizer decides whether the index is actually cheaper.

Index for Range Queries

Indexes are particularly useful for range queries.

Example:

```
SELECT *  
FROM employees  
WHERE salary  
BETWEEN 50000 AND 100000;
```

Or:

```
WHERE  
created_at >= '2026-01-01'  
AND  
created_at < '2026-02-01';
```

B+ Trees are good at this because leaf entries are ordered.

Hash Index

Another index type is:

| **Hash index**

Conceptually:

```
hash(key)
  ↓
bucket
  ↓
row
```

Excellent for:

```
WHERE email='x@example.com'
```

because it's designed for equality lookups.

But traditional hash indexes aren't suitable for normal ordered range queries like:

```
WHERE age>20
```

because hash values don't preserve the ordering of the original keys.

49. B-Tree vs Hash

B-Tree	Hash
Equality search	Equality search
Range queries	Generally poor for range
ORDER BY can benefit	Not naturally ordered
General-purpose	Specialized

For most general relational workloads, B-tree/B+ tree indexes are the common default.

50. Bitmap Index

Bitmap indexes are particularly useful in some analytical/data-warehouse systems.

Example:

```
Gender
-----
Male
Female
```

or:

```
Status
-----
Active
Inactive
```

Low-cardinality columns can be represented efficiently using bitmaps.

They are common in some OLAP systems.

But don't confuse them with the usual OLTP B-tree index.

51. Full-Text Index

If you're searching text:

```
"database indexing tutorial"
```

a normal B-tree isn't necessarily the best tool.

Full-text search indexes are designed for:

```
words
terms
documents
```

rather than exact equality/range lookups.

52. Spatial Index

For geographical data:


```
latitude
longitude
```

specialized spatial indexes can help queries such as:

Find restaurants within 5 km.

Common spatial indexing approaches include structures such as:

```
R-tree
GiST-based spatial indexes
```

depending on the DBMS.

53. Partial / Filtered Index

Suppose you have:

```
10 million users
```

but only:

```
100,000 active users
```

and your application constantly queries:

```
WHERE status='active'
```

Some databases support a partial/filtered index.

PostgreSQL example:

```
CREATE INDEX idx_active_users ON users(email) WHERE status='active';
```

Now the index only contains qualifying rows.

Benefits:

```
Smaller index
↓
Less storage
↓
Less maintenance
↓
Potentially faster queries
```

54. Functional / Expression Index

Suppose:

```
WHERE LOWER(email)='kinza@example.com'
```

You can sometimes create:

```
CREATE INDEX idx_lower_email ON users(LOWER(email));
```

Now the index corresponds to the expression used by the query.

55. Indexing Every Column?

Interviewer:

| "Why not create indexes on every column?"

Answer:

| Because indexes consume storage and increase the cost of INSERT, UPDATE, and DELETE operations. They also require maintenance, and the optimizer may not benefit from indexes on low-selectivity or rarely queried columns.

Excellent interview answer.

56. Too Many Indexes

Suppose:

```
Table
├─ index A
├─ index B
├─ index C
├─ index D
├─ index E
└─ index F
```

Every write may need to update multiple indexes.

So:

```
More indexes
  ↓
Better potential reads
  ↓
More expensive writes
  ↓
More storage
```

It's a trade-off.

57. Index vs Primary Key

These are **not the same thing**.

Primary Key

A database constraint that identifies each row uniquely.

Example:

```
idINTPRIMARYKEY
```

Index

A data structure used to speed up access.

Example:

```
CREATE INDEX idx_users_name ON users(name);
```

A primary key is typically backed by a unique index, but conceptually:

```
PRIMARY KEY  $\neq$  INDEX
```

They serve different purposes.

58. Primary Key vs Unique Key

Primary Key

- uniquely identifies rows
- cannot be NULL
- typically one primary key constraint per table

UNIQUE

- enforces uniqueness
- NULL behavior depends on DBMS
- multiple UNIQUE constraints can exist

Both are commonly backed by indexes.

59. Dense vs Sparse Index

This is more common in database theory/interviews.

Dense Index

Has an index entry for every search-key value/record, depending on the indexing model.

Conceptually:

```
1 → row  
2 → row  
3 → row  
4 → row
```

Sparse Index

Has entries for only some keys, often relying on ordered data.

Conceptually:

```
1   → block  
10  → block  
20  → block  
30  → block
```

Sparse indexes generally require the underlying data to be ordered appropriately.

60. Primary Index vs Secondary Index

Primary index

Associated with the primary/ordering key in classical database theory.

Secondary index

An additional index built on another attribute.

Example:

```
Primary:
id

Secondary:
email
name
department
```

Terminology can vary between database systems, so be careful.

61. Index Scan vs Index Seek

This is especially relevant in SQL Server terminology.

Index Seek

Database navigates directly to relevant portions of the index.

Usually highly selective.

```
Find key
↓
Jump to relevant location
```

Index Scan

Database scans many/all entries in the index.

```
Index entry 1
Index entry 2
Index entry 3
...
```

An index scan isn't necessarily bad.

If the index is smaller than the table and contains all needed information, scanning the index can still be efficient.

62. Index Seek vs Table Scan

Interview question:

| Which is faster?

Correct answer:

| It depends.

A seek is often efficient for selective queries.

But if you're retrieving most of the table:

```
SELECT * FROM users;
```

a table scan may be better.

Don't memorize:

```
Index = always fast  
Table scan = always bad
```

That's incorrect.

63. SARGability

More advanced but valuable for backend interviews.

SARG means:

| Search ARGument.

A predicate is generally considered **SARGable** when the database can efficiently use an index to search it.

Good:

```
WHERE created_at>='2026-01-01'
```

Potentially problematic:

```
WHERE YEAR(created_at)=2026
```

because you're applying a function to the indexed column.

Instead:

```
WHERE created_at>='2026-01-01'AND created_at<'2027-01-01'
```

This can be much more index-friendly.

64. Why **SELECT *** Can Be Bad for Index Performance

Suppose you have:

```
CREATE INDEX idx_users_emailON users(email);
```

Query:

```
SELECT * FROM users WHERE email='x';
```

The index may locate the row, but the database may then need to fetch the complete table row.

If you only need:

```
SELECT email FROM users WHERE email='x';
```

the index itself may contain everything required.

That's where covering/index-only strategies can help.

65. Example: Designing Indexes

Suppose:

```
CREATE TABLE orders (  
  id BIGINT PRIMARY KEY,  
  user_id BIGINT,  
  status VARCHAR(20),  
  created_at TIMESTAMP,  
  total DECIMAL(10,2)  
);
```

Application frequently runs:

Query 1

```
SELECT * FROM orders  
WHERE user_id=100;
```

Potential index:

```
CREATE INDEX idx_orders_user  
ON orders(user_id);
```

Query 2

```
SELECT * FROM orders  
WHERE user_id=100  
AND status='pending';
```

Potential composite index:

```
CREATE INDEX idx_orders_user_status ON orders(user_id, status);
```

Query 3

```
SELECT * FROM orders  
WHERE user_id=100  
ORDER BY created_at DESC;
```

Potential index:

```
CREATE INDEX idx_orders_user_created  
ON orders(user_id, created_at);
```

Depending on DBMS, you may also specify sort direction.

66. Indexing Strategy - Interview-Level Thinking

Don't say:

| "I'll add an index to every WHERE column."

Instead:

Step 1

Look at the actual workload.

Which queries run frequently?

Step 2

Look at:

```
WHERE  
JOIN  
ORDER BY  
GROUP BY
```

Step 3

Check selectivity/cardinality.

Step 4

Consider composite indexes.

Step 5

Check execution plans:

```
EXPLAIN
```

Step 6

Measure before/after.

Step 7

Remove redundant/unhelpful indexes.

This is how a professional approaches indexing.

67. Redundant Indexes

Suppose you have:

```
INDEX(a)
```

and:

```
INDEX(a,b)
```

The second index may make the first partly redundant for some queries, but whether you can remove the first depends on the DBMS and workload.

Don't automatically delete it.

Analyze:

```
Query workload  
+  
execution plans  
+  
write overhead
```

68. Index Maintenance

Indexes aren't "set and forget."

Over time, depending on DBMS/storage engine, you may need to deal with:

- bloat
- fragmentation
- stale statistics
- rebuilding/reorganizing indexes
- vacuuming/analyzing
- unused indexes

Different databases handle this differently.

69. Statistics

The optimizer relies heavily on statistics.

For example:

```
Column:
status

Statistics:
active = 95%
inactive = 5%
```

The optimizer can use this information to estimate how many rows a condition will return.

If statistics are inaccurate:

```
Wrong estimate
↓
Potentially bad execution plan
```

So indexing and query optimization aren't just about indexes themselves.

70. Indexes and Transactions

Indexes participate in database modifications and therefore are affected by transaction semantics.

For:

```
BEGIN;INSERT ...UPDATE ...COMMIT;
```

the database maintains the necessary index structures as part of the transaction.

If the transaction rolls back, those changes are rolled back as well.

71. Indexes and Concurrency

With many concurrent writes:

```
User A → INSERT
User B → UPDATE
```

```
User C → DELETE  
User D → INSERT
```

indexes introduce additional structures that must be maintained safely.

Modern databases use sophisticated locking/MVCC/concurrency mechanisms to handle this.

For interviews, the key idea is:

Indexes improve reads but can increase write contention and maintenance work.

72. What Is a Composite Index Used For?

Interviewer:

"Why would you use a composite index?"

Answer:

When queries commonly filter, join, sort, or otherwise access data using multiple columns together. A composite index can allow the database to efficiently use the combined ordering of those columns.

Example:

```
CREATE INDEX idx_orders_user_status  
ON orders(user_id, status);
```

Useful for:

```
WHERE user_id= ? AND status= ?
```

74. What Is the Leftmost Prefix Rule?

Answer:

For a B-tree composite index such as `(A, B, C)`, the index can generally be used efficiently for predicates involving the leading column `A`, or `A+B`, or `A+B+C`. A query using only `B` and `C` generally cannot use the index as effectively because it skips the leading key.

Memorize this.

PART 21 - ACID

Absolutely memorize this.

Transactions should provide:

A - Atomicity

All operations succeed or none do.

Example bank transfer:

```
A → B
$100 deducted
$100 added
```

Both should happen.

C - Consistency

Database moves from one valid state to another valid state.

Constraints remain satisfied.

I - Isolation

Concurrent transactions shouldn't incorrectly interfere with each other.

D - Durability

Once committed, data survives failures such as a crash.

75. Transaction

A transaction is a logical unit of database work.

```
BEGIN;
```

```
UPDATE accounts  
SET balance = balance - 100  
WHERE id = 1;
```

```
UPDATE accounts  
SET balance = balance + 100  
WHERE id = 2;
```

```
COMMIT;
```

If something goes wrong:

```
ROLLBACK;
```

PART 22 - COMMIT / ROLLBACK

COMMIT

Permanently commit transaction changes.

ROLLBACK

Undo uncommitted changes.

76. SAVEPOINT

Allows partial rollback.


```
BEGIN;  
UPDATE accounts  
SET balance= balance-100  
WHERE id=1;  
SAVEPOINT s1;  
UPDATE accounts  
SET balance= balance+100  
WHERE id=2;  
ROLLBACK TO s1;  
COMMIT;
```

PART 23 - TRANSACTION ISOLATION

Important.

Know these four standard levels:

```
READ UNCOMMITTED  
READ COMMITTED  
REPEATABLE READ  
SERIALIZABLE
```

Dirty Read

Transaction A reads data that Transaction B changed but hasn't committed.

```
B changes  
↓  
A reads  
↓  
B rolls back
```

A saw data that never became committed.

77. Non-repeatable Read

Transaction A reads a row.

Transaction B changes and commits it.

Transaction A reads again.

Gets a different value.

78. Phantom Read

Transaction A executes:

```
SELECT * FROM users
WHERE age > 20;
```

Then another transaction inserts a matching row.

A executes again and sees an additional row.

That's a phantom.

79. Isolation Table

Conceptually:

Level	Dirty Read	Non-repeatable	Phantom
Read Uncommitted	Possible	Possible	Possible
Read Committed	No	Possible	Possible
Repeatable Read	No	No	DB-dependent details
Serializable	No	No	No

PostgreSQL detail

PostgreSQL's `READ UNCOMMITTED` behaves like `READ COMMITTED`, and PostgreSQL's `REPEATABLE READ` is stronger than the minimum SQL standard requirement and prevents phantom reads under its MVCC model.

For an interview, saying simply "Repeatably Read always prevents every phantom in every DB" is too broad.

PART 24 - MVCC

Important for PostgreSQL.

MVCC:

| Multi-Version Concurrency Control.

Instead of forcing every reader to wait for writers, databases can maintain multiple row versions.

Conceptually:

```
Transaction A
  ↓
old version

Transaction B
  ↓
new version
```

This allows good concurrency.

PostgreSQL heavily relies on MVCC.

PART 25 - LOCKS

Two broad concepts:

Shared Lock

Used for reading.

Multiple transactions can potentially hold shared locks simultaneously.

Exclusive Lock

Used for modifications.

Conflicts with other incompatible locks.

80. Deadlock

Example:

```
Transaction A locks Row 1  
Transaction B locks Row 2  
  
A wants Row 2  
B wants Row 1
```

Both wait.

That's a deadlock.

How to prevent deadlocks?

Use consistent lock ordering.

For example:

```
Always lock lower ID first.
```

Also:

- keep transactions short
 - avoid unnecessary locks
 - don't perform slow external work inside transactions
 - retry safely when appropriate
-

PART 26 - DELETE vs TRUNCATE vs DROP

Very common.

DELETE

```
DELETE FROM users  
WHERE id=1;
```

- removes rows
- can use WHERE
- transactional behavior depends on DB, PostgreSQL supports rollback
- row-level delete processing

TRUNCATE

```
TRUNCATE TABLE users;
```

- removes all rows
- cannot use WHERE
- generally faster for clearing the entire table
- PostgreSQL supports transactional TRUNCATE

DROP

```
DROP TABLE users;
```

Removes:

```
table  
structure  
data
```

PART 27 - VIEW

A view is a stored query presented like a virtual table.

```
CREATE VIEW active_users AS SELECT * FROM users WHERE active=true;
```

Then:

```
SELECT * FROM active_users;
```

81. Materialized View

Unlike a normal view, it stores the query result physically.

Useful for expensive read-heavy queries.

But data must be refreshed.

PostgreSQL:

```
REFRESH MATERIALIZED VIEW my_view;
```

PART 28 - STORED PROCEDURE / FUNCTION

Database-side reusable logic.

PostgreSQL supports functions and procedures.

Interview idea:

Use database-side logic selectively; don't automatically put all business logic inside stored procedures.

General SQL Difference Matrix

While PostgreSQL 11+ brought procedures to Postgres, these core rules apply across almost all relational databases (SQL Server, Oracle, MySQL, PostgreSQL).

Feature	Stored Procedure (CREATE PROCEDURE)	Function (CREATE FUNCTION)
Primary Purpose	Execute business logic, perform data modifications (DML), administrative tasks.	Compute and return a single value or table; perform calculations.
Invocation	Called via <code>CALL proc_name()</code> (Postgres/MySQL) or <code>EXEC proc_name</code> (SQL Server).	Called inside SQL statements: <code>SELECT func()</code> , <code>WHERE func() = 1</code> .
Return Value	Optional. Returns zero, one, or multiple result sets/OUT parameters.	Mandatory. Must return a single value (scalar) or a table result set.
Transaction Control	Supports <code>COMMIT</code> and <code>ROLLBACK</code> within the code.	Cannot contain transaction statements (<code>COMMIT</code> / <code>ROLLBACK</code>).
Side Effects / DML	Can freely run <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , and <code>DDL</code> statements.	Strictly limited or forbidden from modifying table data (varies by engine; discouraged globally).
Try-Catch / Error Handling	Allowed (using <code>TRY...CATCH</code> in T-SQL or <code>EXCEPTION</code> blocks in PL/pgSQL).	Highly restricted or not allowed in standard SQL syntax.
Usage in Queries	Cannot be called inside a <code>SELECT</code> statement.	Can be embedded directly inside <code>SELECT</code> , <code>JOIN</code> , <code>WHERE</code> , and <code>HAVING</code> clauses.

2. Top Interview Questions (Theory & Scenario)

Q1. Why can't a Function execute a `COMMIT` or `ROLLBACK` ?

- **Answer:** SQL queries (like `SELECT`) are expected to be deterministic and safe to run in parallel. If a function called inside a `SELECT` statement were allowed to arbitrarily `COMMIT` or `ROLLBACK` , it would break the transaction state of the parent query, lead to lock escalation issues, and ruin ACID isolation guarantees.

Q2. What are `IN` , `OUT` , and `INOUT` parameters?

- **Answer:**
 - **`IN` (Default):** Passes a read-only value into the procedure/function.

- **OUT** : Passes an empty variable into the procedure; the procedure assigns and returns a value back to the caller.
- **INOUT** : Passes an initial value into the procedure, which can modify and return the updated value.

Q3. Can a Function call a Stored Procedure? Can a Stored Procedure call a Function?

- **Answer:**
 - **Procedure $\rightarrow$ Function: YES.** A procedure can call functions inside its statements.
 - **Function $\rightarrow$ Procedure: NO** (in standard databases). Because procedures can modify state and handle transactions, calling them from inside a function would violate the side-effect-free contract of functions.

Q4. How do Stored Procedures prevent SQL Injection?

- **Answer:** Stored procedures separate the **query structure** from the **user data** by using parameterized parameters. When a query is parameterized, the database treats input parameters strictly as data values, never as executable SQL code. *(Note: If dynamic SQL inside a procedure is concatenated manually without parameter binding, it remains vulnerable).*

Q5. What is the performance impact of Scalar User-Defined Functions (UDFs) in queries?

- **Answer:** Calling a scalar UDF inside a **SELECT** clause across a large dataset (e.g., `SELECT format_name(id) FROM users`) causes a **Row-By-Row processing (RBAR)** pattern. The database context-switches from the SQL engine to the procedural language engine for *every single row*, leading to severe performance bottlenecks. **Optimization:** Rewrite scalar functions into Inline Table-Valued Functions (iTVFs) or SQL **JOIN**s.

3. Top Interview Practical Coding Questions

Scenario 1: Returning Multiple Rows from a Function

Question: Write a function that accepts a Department ID and returns all employees in that department.

PostgreSQL Implementation:

SQL

```
CREATE OR REPLACE FUNCTION get_dept_employees(p_dept_id INT)
RETURNS TABLE (
    emp_id INT,
    emp_name VARCHAR,
    emp_salary NUMERIC
)
AS
    SELECT employee_id, name, salary
    FROM employees
    WHERE department_id = p_dept_id;

-- Calling the function like a table:
SELECT * FROM get_dept_employees(10);
```

Scenario 2: Error Handling inside a Stored Procedure

Question: Write a procedure to transfer money between two bank accounts. Handle errors so that if the source account has insufficient funds, no changes are saved.

PL/pgSQL Implementation:

SQL

```
CREATE OR REPLACE PROCEDURE transfer_funds(
    p_sender_id INT,
    p_receiver_id INT,
    p_amount NUMERIC
)
AS
DECLARE
```

```

    v_sender_balance NUMERIC;
BEGIN
    -- 1. Check sender balance with pessimistic locking
    SELECT balance INTO v_sender_balance
    FROM accounts
    WHERE account_id = p_sender_id
    FOR UPDATE;

    IF v_sender_balance < p_amount THEN
        RAISE EXCEPTION 'Insufficient balance in account %',
p_sender_id;
    END IF;

    -- 2. Deduct from sender
    UPDATE accounts
    SET balance = balance - p_amount
    WHERE account_id = p_sender_id;

    -- 3. Add to receiver
    UPDATE accounts
    SET balance = balance + p_amount
    WHERE account_id = p_receiver_id;

    -- Explicit commit on success
    COMMIT;

EXCEPTION WHEN OTHERS THEN
    -- Rollback on any failure/exception
    ROLLBACK;
    RAISE NOTICE 'Transaction aborted: %', SQLERRM;
END;

-- Calling the procedure:
CALL transfer_funds(101, 102, 500.00);

```

PART 29 - TRIGGER

Automatically executes when an event occurs.

Example:

```
INSERT
UPDATE
DELETE
```

Could be used to maintain audit records.

Example:

```
User updated
    ↓
Trigger
    ↓
Audit table
```

But excessive trigger usage can make application behavior harder to reason about.

A **Trigger** is a special database object that automatically executes (fires) when a specific event (such as `INSERT`, `UPDATE`, or `DELETE`) occurs on a table.

Here are simple, clean code examples demonstrating how triggers work in both **PostgreSQL** and **MySQL / SQL Server**.

The Scenario

Suppose you have an `orders` table and an `audit_logs` table. Whenever a new order is placed, you want a trigger to automatically insert an entry into the audit log.

1. Setup Tables

SQL

```
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
```

```

        customer_name VARCHAR(100),
        total_amount NUMERIC(10, 2)
    );

CREATE TABLE audit_logs (
    log_id SERIAL PRIMARY KEY,
    action VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Standard MySQL Syntax

In MySQL, you define the trigger action directly inside the trigger statement without needing a separate function.

SQL

```

DELIMITER //

CREATE TRIGGER trigger_after_order_insert
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
    INSERT INTO audit_logs (action)
    VALUES (CONCAT('New order created with ID: ', NEW.order_id));
END; //

DELIMITER ;

```

2. Testing the Trigger

Insert a row into `orders`:

SQL

```
INSERT INTO orders (customer_name, total_amount)
VALUES ('Alice', 149.99);
```

Check the `audit_logs` table:

SQL

```
SELECT * FROM audit_logs;
```

Output:

log_id	action	created_at
1	New order created with ID: 1	2026-09-01 10:40:15

Core Trigger Variables

When writing triggers, you rely on two special keyword record types:

Variable	Description	Available In
<code>NEW</code>	Refers to the new record being inserted or updated.	<code>INSERT</code> , <code>UPDATE</code>
<code>OLD</code>	Refers to the original record before modification or deletion.	<code>UPDATE</code> , <code>DELETE</code>

PART 30 - SQL vs NOSQL

Very common.

SQL

Examples:

- PostgreSQL
- MySQL
- SQL Server

Good when you need:

- relationships
 - strong consistency
 - complex queries
 - transactions
 - structured data
-

NoSQL

Examples:

- MongoDB
- DynamoDB
- Redis

Useful when you need:

- flexible schemas
- certain high-scale access patterns
- specialized data models

Don't say:

| NoSQL is faster than SQL.

That's an oversimplification.

The right database depends on workload and access patterns.

82. When would you choose PostgreSQL?

Strong backend answer:

| I'd choose PostgreSQL when the application has relational data, complex queries, transactional requirements, strong consistency requirements, and needs features such as foreign keys, indexing, constraints, JSONB, and mature SQL support.

PART 31 - RELATIONSHIPS

One-to-One

```
User → Profile
```

One user has one profile.

One-to-Many

```
User → Orders
```

One user:

```
many orders
```

Foreign key goes on the "many" side.

```
orders.user_id
```

Many-to-Many

Example:

```
students ↔ courses
```

Need junction table:

```
student_courses  
-----
```

```
student_id  
course_id
```

Usually:

```
PRIMARYKEY (student_id, course_id)
```

PART 32 - DATABASE DESIGN QUESTION

Interviewer:

| Design database for an e-commerce system.

You might create:

```
users  
products  
orders  
order_items  
payments  
categories
```

Relationships:

```
users  
├── orders  
│   ├── order_items  
│   │   └── products
```

Important:

Don't store:

```
product1  
product2
```


product3

inside `orders` .

Use normalized relationships.

PART 33 - TRANSACTION IN BACKEND

Very important for your backend interviews.

Suppose an API:

POST /orders

needs to:

1. create order
2. create order items
3. reduce inventory
4. create payment record

These may need one transaction depending on the business consistency requirements.

Concept:

```
BEGIN
↓
create order
↓
create items
↓
update inventory
↓
create payment record
↓
COMMIT
```

If something fails:

ROLLBACK

PART 34 - N+1 QUERY PROBLEM

Very common in backend interviews.

Suppose:

1 query → get 100 users

Then:

100 queries → get orders for each user

Total:

101 queries

That's N+1.

Instead, use:

- JOIN
- eager loading
- batching
- appropriate ORM features

PART 35 - PAGINATION

Offset pagination

```
SELECT * FROM users
ORDER BY id
LIMIT 20
OFFSET 100;
```

Easy but can become inefficient for large offsets and can behave poorly when rows are changing.

83. Cursor Pagination

Instead of:

```
page=1000
```

use:

```
last_seen_id=5000
```

Example:

```
SELECT * FROM users
WHERE id>5000
ORDER BY id
LIMIT 20;
```

Often better for large datasets.

PART 36 - UPSERT

Very useful in PostgreSQL.

Insert if not exists, otherwise update.

```
INSERT INTO users (email, name)
VALUES ('a@gmail.com', 'Ali')
ON CONFLICT (email)
DO UPDATE
SET name= EXCLUDED.name;
```

PART 37 - CASCADE

Foreign keys can define actions.

```
FOREIGN KEY (user_id)
REFERENCES users(id)
ON DELETE CASCADE
```

If parent is deleted:

```
User deleted
↓
related records deleted
```

Other actions:

```
CASCADE
RESTRICT
NO ACTION
SET NULL
SET DEFAULT
```

Use cascade carefully-especially for important data.

PART 38 - SQL INJECTION

Security interview question.

Bad:

```
const query=`SELECT * FROM users WHERE email = '${email}'`;
```

Attacker-controlled input can alter SQL.

Use parameterized queries.

Example concept:

```
db.query("SELECT * FROM users WHERE email = $1",  
  [email]  
);
```

With ORMs, use their parameterization mechanisms and avoid constructing raw SQL from untrusted strings.

84. Why ORMs?

You have worked with SQLAlchemy, so know this.

ORM:

| Object Relational Mapping.

Maps application objects to database tables.

Example:

```
Python User object  
  ↓  
users table
```

Benefits:

- less boilerplate
- parameterization
- models
- relationships

- migrations/tooling depending on ORM
- easier application development

But:

ORM does NOT eliminate the need to understand SQL.

PART 40 - DATABASE CONNECTION POOL

Important backend question.

Creating a database connection is relatively expensive.

Instead of:

```
Request
↓
create connection
↓
query
↓
destroy connection
```

use:

```
Connection Pool
├─ connection 1
├─ connection 2
├─ connection 3
└─ connection 4
```

Request borrows connection:

```
request
↓
pool
↓
connection
↓
query
```

↓
return connection

Benefits:

- lower connection overhead
- controlled concurrency
- better performance

PART 41 - DATABASE SCALING

Vertical Scaling

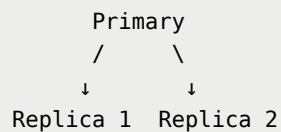
Increase:

CPU
RAM
Storage

Horizontal Scaling

Add more database servers.

85. Read Replicas



Writes:

Primary

Reads:

Replicas

Useful for read-heavy systems.

But replication can introduce lag.

86. Replication

Copies data from one database server to another.

Types/concepts:

- primary/replica
- synchronous
- asynchronous

87. Sharding

Split data across multiple database nodes.

Example:

```
Users A-H → DB1  
Users I-P → DB2  
Users Q-Z → DB3
```

Useful at very large scale.

But introduces complexity:

- cross-shard queries
- transactions
- rebalancing
- shard key design

PART 42 - PARTITIONING

Different from sharding.

Partitioning divides a table into partitions, often within the same database system.

Example:

```
orders_2024  
orders_2025  
orders_2026
```

Range partitioning:

```
date < 2025  
date >= 2025
```

Can improve management and query performance for suitable workloads.

PART 43 - MOST ASKED THEORY QUESTIONS

You should be able to answer these without hesitation.

Q: Primary key vs foreign key?

Primary key uniquely identifies a row.

Foreign key references a key in another table and maintains referential integrity.

Q: WHERE vs HAVING?

WHERE filters rows before grouping.

HAVING filters groups after grouping.

Q: DELETE vs TRUNCATE?

DELETE removes rows and can filter using WHERE.

TRUNCATE clears the table efficiently and doesn't support WHERE.

Q: DROP vs TRUNCATE?

DROP removes the table itself.

TRUNCATE removes its rows but keeps table structure.

Q: UNION vs UNION ALL?

UNION

removes duplicates.

UNIONALL

keeps duplicates and is generally cheaper because it doesn't perform duplicate elimination.

Q: INNER JOIN vs LEFT JOIN?

INNER JOIN → matching records only.

LEFT JOIN → all left records + matching right records.

Q: COUNT(*) vs COUNT(column)?

COUNT(*) counts rows.

COUNT(column) counts non-null values.

Q: What is normalization?

Organizing relational data to reduce redundancy and anomalies.

Q: What is denormalization?

Intentionally introducing redundancy to optimize certain read workloads.

Q: What is an index?

A data structure that improves data lookup for suitable queries, at the cost of storage and write/maintenance overhead.

Q: Why not index everything?

Indexes consume space and make writes more expensive.

Q: What is a transaction?

A logical unit of database work treated as one operation.

Q: What is ACID?

Atomicity
Consistency
Isolation
Durability

Q: What is deadlock?

Two or more transactions wait for resources held by each other.

Q: What is a dirty read?

Reading uncommitted changes from another transaction.

Q: What is a phantom read?

Repeated query returns a changed set of rows because another transaction inserted/deleted matching rows.

Q: What is MVCC?

Multi-Version Concurrency Control; allows transactions to work with appropriate row versions to improve concurrent access.

PART 44 - VERY COMMON SQL CODING ROUND

These are the ones I'd specifically practice on a whiteboard/editor.

Easy

1. Find all employees with salary > 50k

```
SELECT * FROM employees  
WHERE salary > 50000;
```

2. Sort employees by salary

```
SELECT * FROM employees  
ORDER BY salary DESC;
```

3. Find unique departments

```
SELECT DISTINCT department  
FROM employees;
```

4. Count employees

```
SELECT COUNT(*)  
FROM employees;
```

5. Average salary

```
SELECT AVG(salary)  
FROM employees;
```

Medium

6. Count employees by department

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department;
```

7. Departments with > 5 employees

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department  
HAVING COUNT(*) > 5;
```

8. Second highest salary

```
SELECT MAX(salary)  
FROM employees  
WHERE salary <  
(SELECT MAX(salary) FROM employees  
);
```

9. Duplicate emails

```
SELECT email, COUNT(*)  
FROM users  
GROUP BY email  
HAVING COUNT(*) > 1;
```

10. Users with no orders

```
SELECT u.*  
FROM users u  
LEFT JOIN orders o  
ON u.id = o.user_id  
WHERE o.id IS NULL;
```

Hard

11. Top 3 salaries per department

Use:

```
DENSE_RANK()  
WITH RankedSalaries AS (  
    SELECT  
        department_id,  
        employee_id,  
        first_name,  
        last_name,  
        salary,  
        DENSE_RANK() OVER (  
            PARTITION BY department_id  
            ORDER BY salary DESC  
        ) AS salary_rank  
    FROM  
        employees  
)  
SELECT  
    department_id,  
    employee_id,  
    first_name,  
    last_name,  
    salary  
FROM
```

```

RankedSalaries
WHERE
    salary_rank <= 3
ORDER BY
    department_id ASC,
    salary DESC;

```

12. Running total

```

SELECT date,
       amount,
       SUM(amount) OVER (ORDER BY date
                        ) AS running_total
FROM transactions;

```

13. Latest order for each user

```

SELECT * FROM
(
    SELECT
        o.*,
        ROW_NUMBER() OVER (
            PARTITION BY user_id
            ORDER BY created_at DESC
        ) AS rn
    FROM orders o
) x
WHERE rn=1;

```

14. Employees earning above department average

Use:

```

AVG() OVER (PARTITION BY ...)

```

15. Find consecutive records / gaps

Know:

```
LAG()  
LEAD()  
ROW_NUMBER()
```

These are increasingly common in SQL interviews.

Key Ranking Differences:

Function	Handling of Ties	Output Example (Tied for #2)
<code>DENSE_RANK()</code>	Keeps tie, next rank is sequential	1, 2, 2, 3 (Includes top 3 salary values)
<code>RANK()</code>	Keeps tie, skips next rank	1, 2, 2, 4 (Rank 3 is skipped)
<code>ROW_NUMBER()</code>	Breaks tie arbitrarily	1, 2, 3, 4 (Strictly 3 total rows returned)

PART 45 - LAG / LEAD

LAG

Gets previous row.

```
SELECT date,  
       amount,  
       LAG(amount) OVER (ORDER BY date  
                        ) AS previous_amount  
FROM transactions;
```

LEAD

Gets next row.

```
LEAD(amount) OVER (ORDER BY date)
```

PART 46 - SQL EXECUTION ORDER

Memorize this:

```
FROM
↓
JOIN
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
DISTINCT
↓
ORDER BY
↓
LIMIT
```

This helps explain why aliases generally aren't available in `WHERE`, because `WHERE` logically occurs before `SELECT`.

PART 47 - DATABASE DESIGN INTERVIEW

Interviewer:

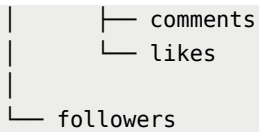
| Design a social media database.

You could propose:

```
users
posts
comments
likes
followers
```

Relationships:

```
users
|
├─ posts
```



Followers is many-to-many:

```
followers
-----
follower_id
following_id
created_at
```

Composite uniqueness:

```
UNIQUE(follower_id, following_id)
```

PART 48 — SOFT DELETE

Instead of:

```
DELETE FROM users;
```

use:

```
deleted_at
```

Example:

id	name	deleted_at
1	Ali	NULL
2	Sara	2026-08-31...

Then active records:

```
WHERE deleted_at IS NULL
```

Useful when you need recovery/auditability.

But it means every query must account for deleted records, often through application conventions or database views.

PART 49 - AUDIT LOG

For security/compliance systems, you may need:

```
audit_logs
-----
id
user_id
action
entity_type
entity_id
old_value
new_value
created_at
```

Example:

```
User 5
updated email
old → a@gmail.com
new → b@gmail.com
```

This is particularly useful in enterprise systems.

PART 50 - MOST IMPORTANT BACKEND QUESTIONS

For your type of backend/SDE interview, I'd expect questions like:

1.

Why is my SQL query slow?

Talk about:

```
EXPLAIN ANALYZE
indexes
query plan
joins
selectivity
unnecessary columns
pagination
statistics
```

2.

Why shouldn't we create indexes on every column?

Answer:

```
storage cost
write overhead
maintenance
optimizer tradeoffs
```

3.

How do you handle 1 million database requests?

Discuss:

```
connection pooling
indexes
query optimization
caching
read replicas
pagination
batching
horizontal scaling
partitioning/sharding where necessary
```

4.

How would you prevent SQL injection?

```
parameterized queries  
prepared statements  
ORM parameterization  
input validation  
least privilege
```

5.

How do transactions work in your backend?

Explain:

```
BEGIN  
↓  
operations  
↓  
COMMIT
```

or:

```
ROLLBACK
```

6.

What happens if your API crashes halfway through a transaction?

A correctly managed transaction should not leave a partially committed unit of work; uncommitted changes are rolled back when the transaction terminates.

7.

How do you prevent duplicate orders?

This is a good system/backend question.

Use a combination of:

```
database UNIQUE constraint
+
idempotency key
+
transaction
```

For example:

```
idempotency_key UNIQUE
```

Then retrying the same request doesn't create another order.

PART 51 - IDEMPOTENCY

Important for backend.

A request is idempotent if repeating it has the same intended effect.

Example:

```
PUT /users/123
```

setting:

```
name = Ali
```

repeatedly should still result in:

```
name = Ali
```

For payments/orders, use idempotency keys to prevent duplicate operations.

PART 52 - DATABASE CONCURRENCY QUESTION

Interviewer:

Two users try to buy the last product simultaneously. What happens?

Naive:

```
User A checks stock = 1
User B checks stock = 1

A buys
B buys
```

Potential overselling.

Solutions can involve:

```
transaction
+
row locking
+
atomic update
```

Example concept:

```
UPDATE products
SET stock = stock-1
WHERE id=10 AND stock>0;
```

Then check affected rows.

If:

```
1 row updated
```

purchase succeeds.

If:

```
0 rows updated
```

stock wasn't available.

This can be an elegant concurrency-safe pattern.

PART 53 - PostgreSQL THINGS YOU SHOULD KNOW

Since you're working with PostgreSQL, revise:

```
PostgreSQL
├─ SERIAL / IDENTITY
├─ UUID
├─ JSONB
├─ ARRAY
├─ ENUM
├─ CTE
├─ Window functions
├─ MVCC
├─ EXPLAIN ANALYZE
├─ B-tree indexes
├─ Partial indexes
├─ Composite indexes
├─ GIN indexes
├─ Transactions
├─ Row-level locking
├─ ON CONFLICT
├─ RETURNING
├─ CTE
└─ VACUUM / ANALYZE
```

54. RETURNING

Very useful in PostgreSQL.


```
INSERT INTO users (name, email)
VALUES ('Ali', 'a@gmail.com')
RETURNING id;
```

You don't need a separate query to retrieve the generated ID.

55. JSONB

PostgreSQL supports JSON data.

```
CREATE TABLE users (
    id BIGINT,
    metadata JSONB
);
```

Can query JSON:

```
SELECT * FROM users
WHERE metadata->>'country'='Pakistan';
```

JSONB also supports indexing, commonly with GIN, for suitable queries.

VACUUM

PostgreSQL uses MVCC, which means old row versions can remain after updates/deletes.

VACUUM helps reclaim/reuse space and maintain table health.

VACUUM ANALYZE also updates planner statistics.

You don't normally manually run vacuum for every query; PostgreSQL's autovacuum handles routine maintenance.